

Conference Paper Title*

*Note: Sub-titles are not captured for <https://ieeexplore.ieee.org> and should not be used

1st Mingyeong Jeong
Computer Science and Engineering
Chungnam National University
Daejeon, South Korea
jmink002@o.cnu.ac.kr

2nd Sungho Lee
Computer Science and Engineering
Chungnam National University
Daejeon, South Korea
eshaj@cnu.ac.kr

Abstract—This document is a model and instructions for L^AT_EX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

Index Terms—component, formatting, style, styling, insert.

well as emerging AI-based type inference techniques enabled by recent advances in large language models.

I. INTRODUCTION

Python has experienced explosive growth since 2018 and has been ranked as the most popular programming language since 2022. This popularity stems from Python’s applicability across a wide range of domains—including networking, data analytics, and artificial intelligence—along with the rapid expansion of AI-related applications. Moreover, its concise, natural-language-like syntax has lowered the barrier to entry, enabling developers to learn and adopt the language with ease.

However, because Python is an interpreted language, many errors surface only at runtime, making its overall stability lower than that of compiled languages. In particular, Python’s dynamic type system leads to frequent runtime type errors, which account for approximately 30% of all reported errors. Consequently, type checking has become essential for improving the reliability of Python programs, and many dynamically typed language communities are increasingly adopting static type systems to enable earlier detection of type-related issues.

Static type systems allow type rules to be verified before execution, thereby enhancing program correctness and reliability. Motivated by this need, the present study aims to evaluate existing type analysis tools and techniques for Python and to identify promising directions for future improvement.

To achieve this, the paper addresses the following research questions (RQs):

- **RQ1** : How can Python’s dynamic nature be preserved while ensuring program stability through type checking?
- **RQ2** : What are the main challenges in type inference and type analysis for Python programs?
- **RQ3** : What technical characteristics and limitations define existing Python type analysis tools?

Additionally, this study examines the current landscape of Python typing technologies, including static analysis tools developed following the introduction of type annotations, as

Identify applicable funding agency here. If none, delete this.